

```

"""
Tetris.py
Simple Tetris game with ghost pieces, next piece preview, increasing speed with score, pause, restart and exit.
Requirements: pip install pygame
"""

import pygame
import random

high_score_file = "data/highscore.txt"

# --- RGB COLORS DEFINITION ---
BLACK = (0, 0, 0)
GRAY = (50, 50, 50)
WHITE = (255, 255, 255)
GOLD = (255, 215, 0)
GHOST_COLOR = (100, 100, 100)
PANEL_COLOR = (111, 111, 111)

# --- UI LAYOUT CONSTANTS ---
# Commands Constants
COMMANDS_START_Y = 70
COMMAND_LINE_HEIGHT = 20
COMMAND_COUNT = 9

# Audio Constants (Y-Coordinates)
AUDIO_START_Y = COMMANDS_START_Y + COMMAND_LINE_HEIGHT * COMMAND_COUNT + 120
MUSIC_CHECKBOX_Y = AUDIO_START_Y
MUSIC_SLIDER_Y = MUSIC_CHECKBOX_Y + 30
SFX_CHECKBOX_Y = MUSIC_CHECKBOX_Y + 50
SFX_SLIDER_Y = SFX_CHECKBOX_Y + 30

# --- CONFIGURATION ---
# Gaming grid Configuration
CELL_SIZE = 30
COLUMNS = 15
ROWS = 30

# Lateral panel Configuration
PANEL_WIDTH = 9 * CELL_SIZE

# Window Configuration
TOTAL_WIDTH = CELL_SIZE * COLUMNS + PANEL_WIDTH
TOTAL_HEIGHT = CELL_SIZE * ROWS
FPS = 60

# Font and Text Configuration
pygame.font.init()
LARGE = pygame.font.SysFont('arial', 50)
MEDIUM = pygame.font.SysFont('arial', 30)
SMALL = pygame.font.SysFont('arial', 20)
COMMAND_LINES = [
    "Commands:",
    "← / A: move left",
    "→ / D: move right",
    "↑ / W: rotate",
    "↓ / S: soft drop",
    "SPACE: hard drop",
    "P : pause",
    "R : restart",
    "M : mute/unmute music",
    "K : mute/unmute SFX",
    "ESC : exit"
]
]
RENDERED_COMMANDS = [SMALL.render(line, True, BLACK) for line in COMMAND_LINES]

# --- TETROMINO CONFIGURATION ---
# Shape colors
COLORS = [
    (0, 255, 255), # I: Hero (Cyan)
    (128, 0, 128), # T: Teewee (Purple)
    (0, 0, 255), # J: Blue Ricky (Blue)
    (255, 165, 0), # L: Orange Ricky (Orange)
    (0, 255, 0), # S: Rhode Island Z (Green)
    (255, 0, 0), # Z: Cleveland Z (Red)
    (255, 255, 0) # O: Smashboy (Yellow)
]

# Shape rotations (stored as string matrices)
SHAPES = [
    # I Shape
    [
        [". . . .",
         "XXXXX",
         ". . . .",
         ". . . ."],
        [". . X.",
         ". . X.",
         ". . X.",
         ". . X."],
    ],
    # T Shape
    [

```

```

        [".X.",
         "XXX",
         "..."],
        [".X.",
         ".XX",
         ".X."],
        ["....",
         "XXX",
         ".X."],
        [".X.",
         "XX.",
         ".X."],
    ],
    # L Shape
    [
        [".X.",
         "XXX",
         "..."],
        [".XX",
         ".X.",
         ".X."],
        ["....",
         "XXX",
         ".X."],
        [".X.",
         ".X.",
         "XX."],
    ],
    # J Shape
    [
        [".X.",
         "XXX",
         "..."],
        [".X.",
         ".X.",
         ".XX"],
        ["....",
         "XXX",
         "X."],
        ["XX.",
         ".X.",
         ".X."],
    ],
    # S Shape
    [
        [".XX",
         "XX.",
         "..."],
        [".X.",
         ".XX",
         ".X."],
    ],
    # Z Shape
    [
        ["XX.",
         ".XX",
         "..."],
        [".X.",
         ".XX",
         ".X."],
    ],
    # O Shape
    [
        ["XX",
         "XX"],
    ]
]

# --- PIECE CLASS ---
# Represents a falling tetromino
class Piece:
    def __init__(self, x, y, shape_type):
        """
        Initialize a new piece with position, shape, and rotation.
        :param x: initial horizontal position.
        :type x: int.
        :param y: initial vertical position.
        :type y: int.
        :param shape_type: identifier of the piece shape.
        :type shape_type: int.
        :returns: None.
        :rtype: None.
        """
        self.x = x
        self.y = y
        self.shape_type = shape_type
        self.rotation = 0
        self.shape_variants = SHAPES[shape_type]
        self.color = COLORS[shape_type]

    @property
    def shape(self):
        """

```

```

        Return the current rotation variant of the piece.
        :param: none.
        :type: none.
        :returns: current shape configuration based on rotation.
        :rtype: list[str].
        """
        return self.shape_variants[self.rotation % len(self.shape_variants)]

def rotate(self):
    """
    Rotate the piece clockwise.
    :param: none.
    :type: none.
    :returns: None.
    :rtype: None.
    """
    self.rotation = (self.rotation + 1) % len(self.shape_variants)

def rotate_back(self):
    """
    Rotate the piece counterclockwise to restore the last rotation.
    :param: none.
    :type: none.
    :returns: None.
    :rtype: None.
    """
    self.rotation = (self.rotation - 1) % len(self.shape_variants)

# --- GLOBAL FUNCTIONS ---
def create_grid(locked_blocks_coordinates = None):
    """
    Create the grid based on locked block positions.
    :param locked_blocks_coordinates: dictionary containing coordinates and colors of locked blocks.
    :type locked_blocks_coordinates: dict.
    :returns: 2D matrix representing the game grid with colors.
    :rtype: list[list[tuple[int, int, int]]].
    """
    # If no arguments are passed (e.g., at the start of the game), initializes an empty dictionary
    locked_blocks_coordinates = locked_blocks_coordinates or {}

    # Creates the grid by checking each coordinate (r, c) in 'locked_blocks_coordinates'.
    # If found, it uses the piece's color; otherwise, it defaults to BLACK.
    return [[locked_blocks_coordinates.get((r, c), BLACK) for c in range(COLUMNS)] for r in range(ROWS)]

def convert_shape_to_positions(piece):
    """
    Convert the current piece shape into grid coordinates.
    :param piece: current falling piece.
    :type piece: Piece.
    :returns: iterable of coordinates occupied by the piece.
    :rtype: generator[tuple[int, int]].
    """
    # Returns the list of all coordinates occupied by the piece by mapping the piece's shape matrix onto the grid
    return ((piece.y + i, piece.x + j) for i, row in enumerate(piece.shape) for j, cell in enumerate(row) if cell == 'X')

def valid_space(piece, grid):
    """
    Check whether a piece is in a valid position inside the grid.
    :param piece: current piece to validate.
    :type piece: Piece.
    :param grid: current grid matrix.
    :type grid: list[list[tuple[int, int, int]]].
    :returns: True if the position is valid, otherwise False.
    :rtype: bool.
    """
    # Creates a set of all coordinates that are currently empty (BLACK)
    accepted_coordinates = {(r, c) for r in range(ROWS) for c in range(COLUMNS) if grid[r][c] == BLACK}

    # Gets the actual grid coordinates for the current piece shape
    actual_piece_coordinates = convert_shape_to_positions(piece)

    # Checks if all blocks of the piece are within boundaries and in empty spaces
    return all(0 <= c < COLUMNS and r < ROWS and (r < 0 or (r, c) in accepted_coordinates) for r, c in actual_piece_coordinates)

def check_lost(locked_blocks_coordinates):
    """
    Determine if the game is lost based on locked block positions.
    :param locked_blocks_coordinates: dictionary of locked blocks.
    :type locked_blocks_coordinates: dict.
    :returns: True if a block reached the top, otherwise False.
    :rtype: bool.
    """
    # Iterates through all the coordinates of the blocks already placed on the board
    for (r, c) in locked_blocks_coordinates:
        # If the row index 'r' is above or at the very first row of the grid, it means there is no more space to play (Game Over)
        if r < 1:
            return True

    # If no blocks are at the top, the game continues
    return False

def clear_rows(grid, locked_blocks_coordinates):
    """

```

```

Remove full rows and update remaining block positions.
:param grid: current grid matrix.
:type grid: list[list[tuple[int, int, int]]].
:param locked_blocks_coordinates: dictionary of locked blocks.
:type locked_blocks_coordinates: dict.
:returns: number of cleared rows and updated locked blocks.
:rtype: tuple[int, dict].
"""
# Initializes the counter for the rows full of locked blocks
rows_cleared = 0

# Scans rows from bottom to top (line by line) to find and delete full rows
for r in range(ROWS - 1, -1, -1):
    # If the row is completely full (no BLACK cells), every block of it is removed from the locked dictionary
    if BLACK not in grid[r]:
        rows_cleared += 1
        for c in range(COLUMNS):
            if (r, c) in locked_blocks_coordinates:
                del locked_blocks_coordinates[(r, c)]

# If rows were cleared, updates the positions of the remaining blocks
if rows_cleared > 0:
    # Initializes a new empty dictionary used for the new locked blocks coordinates
    new_locked_blocks_coordinates = {}

    # Initializes a counter that tracks how many positions down a block must fall
    shift_down = 0

    # Scans again from bottom to top to calculate the downward shift
    for r in range(ROWS - 1, -1, -1):
        # Checks if this row was one of the cleared ones
        if BLACK not in grid[r]:
            shift_down += 1
        else:
            # Moves blocks down by the number of cleared rows below them
            for c in range(COLUMNS):
                if (r, c) in locked_blocks_coordinates:
                    # Adds the block to the new dictionary with updated row (r + shift_down)
                    new_locked_blocks_coordinates[(r + shift_down, c)] = locked_blocks_coordinates[(r, c)]

    return rows_cleared, new_locked_blocks_coordinates

# If no rows were full, returns the original state
return 0, locked_blocks_coordinates

def draw_grid(surface):
    """
    Draw the grid lines on the surface.
    :param surface: pygame surface where the grid is drawn.
    :type surface: pygame.Surface.
    :returns: None.
    :rtype: None.
    """
    # Draws horizontal lines for each row
    for i in range(ROWS):
        pygame.draw.line(surface, GRAY, (0, i * CELL_SIZE), (CELL_SIZE * COLUMNS, i * CELL_SIZE))

    # Draws vertical lines for each column
    for j in range(COLUMNS):
        pygame.draw.line(surface, GRAY, (j * CELL_SIZE, 0), (j * CELL_SIZE, CELL_SIZE * ROWS))

def draw_next_piece(surface, piece, x_offset, y_offset):
    """
    Render the preview of the next piece in the side panel.
    :param surface: pygame surface where the preview is drawn.
    :type surface: pygame.Surface.
    :param piece: next piece to display.
    :type piece: Piece.
    :param x_offset: horizontal offset for drawing.
    :type x_offset: int.
    :param y_offset: vertical offset for drawing.
    :type y_offset: int.
    :returns: None.
    :rtype: None.
    """
    # Initializes the shape for the piece, that will be displayed, and its pixel size
    shape = piece.shape
    piece_cell_size = CELL_SIZE

    # Calculates the maximum width of the shape to center it properly
    shape_width = max(len(row) for row in shape)

    # Calculates starting coordinates to center the piece within the preview panel
    start_x = x_offset + (PANEL_WIDTH - shape_width * piece_cell_size) // 2 - 10
    start_y = y_offset

    # Loops through the shape matrix and draws only the 'X' blocks
    for i, row in enumerate(shape):
        for j, ch in enumerate(row):
            if ch == 'X':
                # Defines the rectangle area for the block
                rectangle_area = pygame.Rect(start_x + j * piece_cell_size, start_y + i * piece_cell_size, piece_cell_size, piece_cell_size)

```

```

        # Fills the block with the piece's color
        pygame.draw.rect(surface, piece.color, rectangle_area)

        # Draws a white border (width 1) around the block for better visibility
        pygame.draw.rect(surface, WHITE, rectangle_area, 1)

def draw_ghost(surface, ghost_piece):
    """
    Draw the ghost piece at its landing position.
    :param surface: pygame surface where the ghost is drawn.
    :type surface: pygame.Surface.
    :param ghost_piece: piece representing the landing position.
    :type ghost_piece: Piece.
    :returns: None.
    :rtype: None.
    """
    # Converts the ghost piece's internal shape into grid (row, column) coordinates
    for (r, c) in convert_shape_to_positions(ghost_piece):
        # Only draws if the block is within the visible part of the grid (row 0 or below)
        if r >= 0:
            # Creates a small square surface for a single block
            ghost_piece_surface = pygame.Surface((CELL_SIZE, CELL_SIZE))

            # Sets the transparency level (0 = invisible, 255 = solid)
            ghost_piece_surface.set_alpha(80)

            # Fills this small surface with the ghost's specific color
            ghost_piece_surface.fill(GHOST_COLOR)

            # Draws (blit) the transparent block onto the main game surface
            surface.blit(ghost_piece_surface, (c * CELL_SIZE, r * CELL_SIZE))

            # Draws a thin white border around the block for better definition
            pygame.draw.rect(surface, WHITE, (c * CELL_SIZE, r * CELL_SIZE, CELL_SIZE, CELL_SIZE), 1)

def draw_game_over(surface, score, high_score):
    """
    Render the game over screen with score information.
    :param surface: pygame surface where the screen is drawn.
    :type surface: pygame.Surface.
    :param score: current game score.
    :type score: int.
    :param high_score: best recorded score.
    :type high_score: int.
    :returns: None.
    :rtype: None.
    """
    # Renders text objects (converts strings into images)
    text1 = LARGE.render("GAME OVER", True, (220, 20, 20))
    text2 = MEDIUM.render(f"Score: {score}", True, WHITE)
    text3 = MEDIUM.render(f"High Score: {high_score}", True, GOLD)
    text4 = SMALL.render("Press R to Restart", True, WHITE)

    # Draws (blit) the text at the center of the screen with vertical spacing
    surface.blit(text1, ((TOTAL_WIDTH - text1.get_width()) // 2, TOTAL_HEIGHT // 2 - 80))
    surface.blit(text2, ((TOTAL_WIDTH - text2.get_width()) // 2, TOTAL_HEIGHT // 2 - 20))
    surface.blit(text3, ((TOTAL_WIDTH - text3.get_width()) // 2, TOTAL_HEIGHT // 2 + 20))
    surface.blit(text4, ((TOTAL_WIDTH - text4.get_width()) // 2, TOTAL_HEIGHT // 2 + 80))

def draw_audio_section(surface, panel_x, music_enabled, music_volume, sfx_enabled, sfx_volume):
    """
    Render audio controls including checkboxes and sliders.
    :param surface: pygame surface where controls are drawn.
    :type surface: pygame.Surface.
    :param panel_x: horizontal position of the panel.
    :type panel_x: int.
    :param music_enabled: state of music.
    :type music_enabled: bool.
    :param music_volume: current music volume.
    :type music_volume: float.
    :param sfx_enabled: state of sound effects.
    :type sfx_enabled: bool.
    :param sfx_volume: current sound effects volume.
    :type sfx_volume: float.
    :returns: None.
    :rtype: None.
    """
    # Initializes the starting y-coordinate to the global constant and the size of checkboxes
    start_y = AUDIO_START_Y
    checkbox_size = 20

    # Configures a list to avoid repeating drawing logic for Music and SFX
    audio_configs = [
        ("Music", music_enabled, music_volume, start_y),
        ("Sound Effects", sfx_enabled, sfx_volume, start_y + 50)
    ]

    for label, enabled, volume, y_position in audio_configs:
        # Draws the checkbox outline
        rect = pygame.Rect(panel_x + 20, y_position, checkbox_size, checkbox_size)
        pygame.draw.rect(surface, BLACK, rect, 2)

```

```

    if enabled:
        # Draws an 'X' inside the checkbox (if enabled)
        pygame.draw.line(surface, BLACK, rect.topleft, rect.bottomright, 2)
        pygame.draw.line(surface, BLACK, rect.topright, rect.bottomleft, 2)

        # Draws the volume slider (line and knob)
        slider_y = y_position + 30
        pygame.draw.rect(surface, BLACK, (panel_x + 20, slider_y, 120, 4))
        knob_x = panel_x + 20 + int(120 * volume)
        pygame.draw.circle(surface, BLACK, (knob_x, slider_y + 2), 6)

    # Renders the label text using the SMALL font
    surface.blit(SMALL.render(label, True, BLACK), (panel_x + 45, y_position - 2))

def draw_window(surface, grid, score, high_score, next_piece, paused, music_enabled, sfx_enabled, music_volume, sfx_volume):
    """
    Render the entire game interface.
    :param surface: pygame surface where everything is drawn.
    :type surface: pygame.Surface.
    :param grid: current grid matrix.
    :type grid: list[list[tuple[int, int, int]]].
    :param score: current game score.
    :type score: int.
    :param high_score: best recorded score.
    :type high_score: int.
    :param next_piece: next piece to display.
    :type next_piece: Piece.
    :param paused: pause state.
    :type paused: bool.
    :param music_enabled: state of music.
    :type music_enabled: bool.
    :param sfx_enabled: state of sound effects.
    :type sfx_enabled: bool.
    :param music_volume: current music volume.
    :type music_volume: float.
    :param sfx_volume: current sound effects volume.
    :type sfx_volume: float.
    :returns: None.
    :rtype: None.
    """
    # Fills the background with black
    surface.fill(BLACK)

    # Defines the starting X position for the side UI panel
    panel_x = CELL_SIZE * COLUMNS

    # Draws the side panel background rectangle
    pygame.draw.rect(surface, PANEL_COLOR, (panel_x, 0, PANEL_WIDTH, TOTAL_HEIGHT))

    # --- GRID RENDERING ---
    # Loops through the grid matrix and draws blocks that are not BLACK
    for i, row in enumerate(grid):
        for j, color in enumerate(row):
            if color != BLACK:
                rect = (j * CELL_SIZE, i * CELL_SIZE, CELL_SIZE, CELL_SIZE)
                # Draws the filled block color
                pygame.draw.rect(surface, color, rect)

                # Draws a white border for visual block separation
                pygame.draw.rect(surface, WHITE, rect, 1)

    # Draws the gray grid lines over the board
    draw_grid(surface)

    # --- UI TEXTS (SCORE & RECORD) ---
    # Uses MEDIUM font for better readability of scores
    surface.blit(MEDIUM.render(f"Score: {score}", True, BLACK), (panel_x + 10, 7))
    surface.blit(MEDIUM.render(f"High Score: {high_score}", True, GOLD), (panel_x + 10, 40))

    # --- COMMANDS LIST ---
    # Renders pre-generated command surfaces
    y = 85
    for text_surface in RENDERED_COMMANDS:
        surface.blit(text_surface, (panel_x + 20, y))
        y += 22

    # --- NEXT PIECE PREVIEW ---
    next_y = TOTAL_HEIGHT - 220
    surface.blit(MEDIUM.render("Next Piece:", True, BLACK), (panel_x + 13, next_y))

    # Calls the preview function to draw the piece shape
    draw_next_piece(surface, next_piece, panel_x + 10, next_y + 50)

    # --- PAUSE OVERLAY ---
    if paused:
        # Uses LARGE font for the main pause message
        text = LARGE.render("PAUSED", True, WHITE)

        # Centers the text automatically within the game grid area
        text_rect = text.get_rect(center = ((CELL_SIZE * COLUMNS) // 2, TOTAL_HEIGHT // 2))
        surface.blit(text, text_rect)

```

```

# --- AUDIO CONTROLS ---
# Helper call to handle the music and SFX UI section
draw_audio_section(surface, panel_x, music_enabled, music_volume, sfx_enabled, sfx_volume)

def init_audio():
    """
    Initialize audio system and load all sounds.
    :param: none.
    :type: none.
    :returns: dictionary containing audio settings and sound objects.
    :rtype: dict.
    """
    # Initializes the pygame mixer with a standard frequency to ensure that sounds are played back with good quality.
    pygame.mixer.init(frequency = 44100)

    # Loads sound effects and music from local files
    rotate = pygame.mixer.Sound("assets/sounds/rotate.wav")
    drop = pygame.mixer.Sound("assets/sounds/drop.wav")
    clear = pygame.mixer.Sound("assets/sounds/clear.wav")
    hard_drop = pygame.mixer.Sound("assets/sounds/lock.wav")
    game_over = pygame.mixer.Sound("assets/sounds/game_over.wav")
    pygame.mixer.music.load("assets/music/tetris_slow.mp3")
    pygame.mixer.music.play(-1) # '-1' -> Play music on loop

    # Default audio settings
    audio = {
        "music_enabled": True,
        "sfx_enabled": True,
        "music_volume": 0.5,
        "sfx_volume": 0.5,
        "current_music": "slow",
        "dragging_music": False,
        "dragging_sfx": False,
        "rotate": rotate,
        "drop": drop,
        "clear": clear,
        "hard_drop": hard_drop,
        "game_over": game_over,
        "sounds": [rotate, drop, clear, hard_drop, game_over]
    }

    # Applies initial volume
    pygame.mixer.music.set_volume(audio["music_volume"])
    for s in audio["sounds"]:
        s.set_volume(audio["sfx_volume"])

    return audio

def update_music(fall_speed, audio):
    """
    Update background music based on fall speed.
    :param fall_speed: current falling speed.
    :type fall_speed: float.
    :param audio: audio configuration dictionary.
    :type audio: dict.
    :returns: None.
    :rtype: None.
    """
    # Decides which music to play based on pieces' fall speed
    if fall_speed > 0.6 and audio["current_music"] != "slow":
        track = "assets/music/tetris_slow.mp3"
        audio["current_music"] = "slow"
    elif 0.3 < fall_speed <= 0.6 and audio["current_music"] != "medium":
        track = "assets/music/tetris_medium.mp3"
        audio["current_music"] = "medium"
    elif 0.1 < fall_speed <= 0.3 and audio["current_music"] != "fast":
        track = "assets/music/tetris_fast.mp3"
        audio["current_music"] = "fast"
    elif fall_speed <= 0.1 and audio["current_music"] != "super_fast":
        track = "assets/music/tetris_super_fast.mp3"
        audio["current_music"] = "super_fast"
    else:
        return

    # Loads and plays the new track
    pygame.mixer.music.load(track)
    pygame.mixer.music.play(-1)
    pygame.mixer.music.set_volume(audio["music_volume"])

def handle_keyboard(event, state, audio):
    """
    Handle keyboard input for gameplay actions.
    :param event: pygame keyboard event.
    :type event: pygame.event.Event.
    :param state: current game state.
    :type state: dict.
    :param audio: audio configuration dictionary.
    :type audio: dict.
    :returns: None.
    :rtype: None.
    """
    # Memorizes the current controlled piece in an object
    piece = state["current_piece"]

```

```

# If 'ESC' is pressed, the game and the window close
if event.key == pygame.K_ESCAPE:
    state["running"] = False

# If 'P' is pressed, the game pause and the music toggle
elif event.key == pygame.K_p:
    state["paused"] = not state["paused"]
    if state["paused"]:
        pygame.mixer.music.pause()
    else:
        pygame.mixer.music.unpause()

# If 'R' is pressed, the game restarts
elif event.key == pygame.K_r:
    state["restart"] = True

# Checks if the game isn't paused or ended
elif not state["paused"] and not state["game_over"]:
    # If Left Arrow or 'A' are pressed, the piece moves to the left
    if event.key in (pygame.K_LEFT, pygame.K_a):
        piece.x -= 1
        if not valid_space(piece, create_grid(state["locked"])):
            piece.x += 1

    # If Right Arrow or 'D' are pressed, the piece moves to the right
    elif event.key in (pygame.K_RIGHT, pygame.K_d):
        piece.x += 1
        if not valid_space(piece, create_grid(state["locked"])):
            piece.x -= 1

    # If Down Arrow or 'S' are pressed, the piece moves down by one (Soft Drop)
    elif event.key in (pygame.K_DOWN, pygame.K_s):
        piece.y += 1
        if not valid_space(piece, create_grid(state["locked"])):
            piece.y -= 1
        elif audio["sfx_enabled"]:
            audio["drop"].play()

    # If Up Arrow or 'K' are pressed, the piece rotates clockwise
    elif event.key in (pygame.K_UP, pygame.K_w):
        piece.rotate()
        if not valid_space(piece, create_grid(state["locked"])):
            piece.rotate_back()
        elif audio["sfx_enabled"]:
            audio["rotate"].play()

    # If Space is pressed, the piece instantly drops (Hard Drop)
    elif event.key == pygame.K_SPACE:
        while True:
            piece.y += 1
            if not valid_space(piece, create_grid(state["locked"])):
                piece.y -= 1
                break
        if audio["sfx_enabled"]:
            audio["hard_drop"].play()
        state["change_piece"] = True

# If 'M' is pressed, the music state changes
elif event.key == pygame.K_m:
    audio["music_enabled"] = not audio["music_enabled"]
    if audio["music_enabled"]:
        pygame.mixer.music.play(-1)
        pygame.mixer.music.set_volume(audio["music_volume"])
    else:
        pygame.mixer.music.stop()

# If 'K' is pressed, the SFX state changes
elif event.key == pygame.K_k:
    audio["sfx_enabled"] = not audio["sfx_enabled"]

def handle_mouse(event, audio):
    """
    Handle mouse input for UI interactions.
    :param event: pygame mouse event.
    :type event: pygame.event.Event.
    :param audio: audio configuration dictionary.
    :type audio: dict.
    :returns: None.
    :rtype: None.
    """
    # Gets the current mouse position (X and Y coordinates)
    mouse_x, mouse_y = pygame.mouse.get_pos()

    # Calculates the start of the side panel
    panel_x = CELL_SIZE * COLUMNS

    # Checks if the user is clicking something with the mouse
    if event.type == pygame.MOUSEBUTTONDOWN:
        # If the user clicks the Music checkbox, the music toggles on or off
        if panel_x + 20 <= mouse_x <= panel_x + 38 and MUSIC_CHECKBOX_Y <= mouse_y <= MUSIC_CHECKBOX_Y + 18:
            audio["music_enabled"] = not audio["music_enabled"]
            if audio["music_enabled"]:

```



```

        pygame.mixer.music.play(-1)
        pygame.mixer.music.set_volume(audio["music_volume"])
    else:
        pygame.mixer.music.stop()

    # If the user clicks the SFX checkbox, the sound effects toggle on or off
    if panel_x + 20 <= mouse_x <= panel_x + 38 and SFX_CHECKBOX_Y <= mouse_y <= SFX_CHECKBOX_Y + 18:
        audio["sfx_enabled"] = not audio["sfx_enabled"]

    # If the user clicks near the Music slider, starts dragging the music volume knob
    if panel_x + 20 <= mouse_x <= panel_x + 140 and MUSIC_SLIDER_Y - 10 <= mouse_y <= MUSIC_SLIDER_Y + 10:
        audio["dragging_music"] = True

    # If the user clicks near the SFX slider, starts dragging the SFX volume knob
    if panel_x + 20 <= mouse_x <= panel_x + 140 and SFX_SLIDER_Y - 10 <= mouse_y <= SFX_SLIDER_Y + 10:
        audio["dragging_sfx"] = True

    # If the user released a mouse button, stops dragging any slider
    elif event.type == pygame.MOUSEBUTTONDOWN:
        audio["dragging_music"] = False
        audio["dragging_sfx"] = False

    # Checks if the user is moving the mouse while keeping clicked something
    elif event.type == pygame.MOUSEMOTION:
        # Updates music volume while dragging the slider
        if audio["dragging_music"]:
            # Calculates volume (0 to 1) based on mouse X position
            audio["music_volume"] = max(0, min(1, (mouse_x - (panel_x + 20)) / 120))
            # Applies the new volume level to the music mixer
            pygame.mixer.music.set_volume(audio["music_volume"])

        # Updates SFX volume while dragging the slider
        if audio["dragging_sfx"]:
            # Calculates volume (0 to 1) based on mouse X position
            audio["sfx_volume"] = max(0, min(1, (mouse_x - (panel_x + 20)) / 120))
            # Applies the new volume to all sound effect objects
            for s in audio["sounds"]:
                s.set_volume(audio["sfx_volume"])

def get_shape():
    """
    Generate a random piece at the starting position.
    :param: none.
    :type: none.
    :returns: newly created piece.
    :rtype: Piece.
    """
    # Selects a random index from the available shape types (0 to 6 for the 7 Tetrominos)
    shape_type = random.randrange(len(SHAPES))

    # Creates and returns the piece at the starting position (Top-Center)
    # x: Calculated to center the piece based on the grid's columns
    # y: Set to -2 so the piece starts slightly above the visible grid
    return Piece(COLUMNS // 2 - 2, -2, shape_type)

def get_ghost_position(current_piece, locked_blocks_coordinates):
    """
    Calculate the landing position of the current piece.
    :param current_piece: active piece.
    :type current_piece: Piece.
    :param locked_blocks_coordinates: dictionary of locked blocks.
    :type locked_blocks_coordinates: dict.
    :returns: piece representing the landing position.
    :rtype: Piece.
    """
    # Creates a 'clone' of the current piece with the same position and type
    ghost_piece = Piece(current_piece.x, current_piece.y, current_piece.shape_type)

    # Ensures the ghost_piece has the same rotation as the active piece
    ghost_piece.rotation = current_piece.rotation

    # Generates a temporary grid to check for collisions
    grid = create_grid(locked_blocks_coordinates)

    # Loops to simulate the piece falling rapidly
    while True:
        # Moves the ghost_piece down by one row
        ghost_piece.y += 1

        # Checks if the new position is invalid (hits a wall, floor, or another piece)
        if not valid_space(ghost_piece, grid):
            # Moves back up to the last valid position and stops the loop (found the landing spot)
            ghost_piece.y -= 1
            break

    return ghost_piece

def load_high_score():
    """
    Load the saved high score from file.
    :param: none.
    :type: none.
    :returns: stored high score value.
    """

```

```

:rtype: int.
"""
try:
    # Opens the high score file in read mode
    with open(high_score_file, "r") as file:
        return int(file.read())
except:
    # If the file is missing or corrupted, returns a default score of 0
    return 0

def save_high_score(score):
    """
    Save the high score to file.
    :param score: score to save.
    :type score: int.
    :returns: None.
    :rtype: None.
    """
    # Opens (or create) the high score file in writing mode
    with open(high_score_file, "w") as file:
        file.write(str(score))

def reset_state():
    """
    Initialize and return the default game state.
    :param: none.
    :type: none.
    :returns: dictionary containing initial state values.
    :rtype: dict.
    """
    return {
        # Keeps the main game loop running
        "running": True,
        # Ensures the game starts unpaused
        "paused": False,
        # Sets game over status to false
        "game_over": False,
        # Resets the restart request flag
        "restart": False,
        # Resets the timer for falling blocks
        "fall_time": 0,
        # Resets the flag that triggers a new piece
        "change_piece": False,
        # Clears all blocks locked on the grid
        "locked": {},
        # Generates the first active piece
        "current_piece": get_shape(),
        # Generates the preview of the next piece
        "next_piece": get_shape(),
        # Resets current score to zero
        "score": 0,
        # Loads the best score from the local file
        "high_score": load_high_score(),
    }

def main():
    """
    Run the main game loop and manage game execution.
    :param: none.
    :type: none.
    :returns: None.
    :rtype: None.
    """
    # Starts Pygame modules
    pygame.init()

    # Sets delay and speed for continuous key presses
    pygame.key.set_repeat(300, 100)

    # Initializes sounds, music, and game state (score, pieces, etc.)
    audio = init_audio()
    state = reset_state()

    # Creates the game window
    display = pygame.display.set_mode((TOTAL_WIDTH, TOTAL_HEIGHT))

    # Sets the window title and icon
    pygame.display.set_caption("Tetris Game")
    pygame.display.set_icon(pygame.image.load("assets/images/icon.jpg"))

    # Instantiates a clock to track time and control FPS
    clock = pygame.time.Clock()

    # Sets the starting fall speed and the fastest possible (in seconds)
    initial_fall_speed = 1
    max_fall_speed = 0.1

    # Loops until the user closes the window game
    while state["running"]:
        # Calculates time passed since the last frame
        dt = clock.tick(FPS) / 1000.0

        # Resets everything if the user wants to restart

```

```

if state["restart"]:
    # Preserves the music's state before restarting
    old_audio = audio
    state = reset_state()
    # Restores audio
    audio = old_audio
    if audio["music_enabled"]:
        pygame.mixer.music.play(-1)
        pygame.mixer.music.set_volume(audio["music_volume"])
    else:
        pygame.mixer.music.stop()

    continue

# Makes the game harder and adjusts music speed as the score increases
current_fall_speed = max(max_fall_speed, initial_fall_speed - (state["score"] / 4000.0))
update_music(current_fall_speed, audio)

# Checks if the game is not paused or ended
if not state["paused"] and not state["game_over"]:
    # Accumulates time for the piece to fall
    state["fall_time"] += dt

# Checks for user inputs (keyboard/mouse)
for event in pygame.event.get():
    # If the user quits the game, the game loop stops
    if event.type == pygame.QUIT:
        state["running"] = False

    # If the user presses any key, the appropriate function processes it
    elif event.type == pygame.KEYDOWN:
        handle_keyboard(event, state, audio)

    # If the user interacts with the mouse in some specific cases, the appropriate function processes them
    elif event.type in (pygame.MOUSEBUTTONDOWN, pygame.MOUSEBUTTONUP, pygame.MOUSEMOTION):
        handle_mouse(event, audio)

if not state["paused"] and not state["game_over"] and state["fall_time"] >= current_fall_speed:
    # Resets the fall time timer and moves the piece down one row automatically
    state["fall_time"] = 0
    state["current_piece"].y += 1

    # If the piece hits something, moves back and prepares to lock
    if not valid_space(state["current_piece"], create_grid(state["locked"])):
        state["current_piece"].y -= 1
        state["change_piece"] = True

# Checks if the current piece is going to be locked
if state["change_piece"]:
    # Adds the current piece positions to the 'locked' dictionary
    for (r, c) in convert_shape_to_positions(state["current_piece"]):
        if r >= 0:
            state["locked"][r, c] = state["current_piece"].color

    # Updates the grid with the new locked piece and removes full lines (if any)
    grid = create_grid(state["locked"])
    rows_cleared, state["locked"] = clear_rows(grid, state["locked"])

    # If some rows are cleared, the clear sound plays
    if rows_cleared > 0 and audio["sfx_enabled"]:
        audio["clear"].play()

    # Updates score
    state["score"] += (100 * rows_cleared) * rows_cleared

    # Switches and generates the next piece, also resets the corresponding flag
    state["current_piece"] = state["next_piece"]
    state["next_piece"] = get_shape()
    state["change_piece"] = False

    # If the blocks reached the top, it's Game Over
    if check_lost(state["locked"]):
        state["game_over"] = True
        pygame.mixer.music.stop()
        if audio["sfx_enabled"]:
            audio["game_over"].play()

# Refreshes the grid state
grid = create_grid(state["locked"])

# Calculates where the piece would land (Ghost)
ghost_piece = get_ghost_position(state["current_piece"], state["locked"])

# Adds the active moving piece to the grid for drawing
for (r, c) in convert_shape_to_positions(state["current_piece"]):
    if r >= 0:
        grid[r][c] = state["current_piece"].color

# Renders everything on the screen
draw_window(display, grid, state["score"], state["high_score"], state["next_piece"], state["paused"],
            audio["music_enabled"], audio["sfx_enabled"],
            audio["music_volume"], audio["sfx_volume"])

```

```
# Draws the transparent ghost piece
draw_ghost(display, ghost_piece)

# If the current score is higher than the record, it is updated and saved to the file
if state["score"] > state["high_score"]:
    state["high_score"] = state["score"]
    save_high_score(state["high_score"])

# If it's Game Over, it is displayed on screen
if state["game_over"]:
    draw_game_over(display, state["score"], state["high_score"])

# Updates the physical monitor display
pygame.display.update()

# Closes the game window and cleans up
pygame.quit()

if __name__ == "__main__":
    main()
```